

Métodos Cuantitativos

Clase 02 - Manejo básico de Python

Unidad I: Estadística y Probabilidades con Python

Repaso de la sesión anterior

- En la primera sesión revisamos los tipos de datos básicos en **Python**: números, texto y valores booleanos.
- También vimos cómo asignar variables, ejecutar operaciones aritméticas y utilizar cuadernos de Jupyter para trabajar de forma interactiva.
- A partir de esa base, hoy incorporaremos herramientas para tomar decisiones, repetir instrucciones y reutilizar código.

Objetivos de la sesión

- Comprender cómo funcionan las **estructuras de control de flujo**.
- Utilizar condicionales `if`, `elif` y `else` para tomar decisiones.
- Trabajar con bucles `for` y `while`.
- Crear funciones simples con parámetros y valores de retorno.

¿Qué veremos hoy?

- Control de flujo y condicionales
- Bucles en Python
- Funciones en Python

¿Qué es el control de flujo?

- Un programa no siempre se ejecuta de manera lineal.
- En muchos problemas debemos decidir entre alternativas o repetir una tarea varias veces.
- Para eso usamos **estructuras de control**, que permiten adaptar la ejecución del código a una condición o patrón.

Condicionales

La estructura básica de un condicional en **Python** es:

```
1 edad = 19
2
3 if edad >= 18:
4     print("Puede votar")
5 else:
6     print("No puede votar")
```

Puede votar

Condicionales con varias alternativas

```
1 nota = 5.3
2
3 if nota >= 5.5:
4     print("Desempeño destacado")
5 elif nota >= 4.0:
6     print("Desempeño suficiente")
7 else:
8     print("Desempeño insuficiente")
```

Desempeño suficiente

Operadores lógicos y relacionales

- Los condicionales suelen apoyarse en operadores relacionales: >, <, >=, <=, ==, !=.
- También podemos combinar condiciones con operadores lógicos como and, or y not.
- Esto resulta útil cuando una decisión depende de más de un criterio.

```
1 ingreso = 850000
2 antigüedad = 3
3
4 if ingreso >= 700000 and antigüedad >= 2:
5     print("Cumple con los requisitos mínimos")
6 else:
7     print("No cumple con los requisitos mínimos")
```

Cumple con los requisitos mínimos

¿Qué veremos hoy?

- Control de flujo y condicionales
- Bucles en Python
- Funciones en Python

Bucle for

- Un bucle `for` recorre una secuencia de elementos.
- Es útil cuando conocemos los datos sobre los que queremos iterar o cuántas repeticiones necesitamos.

```
1 ventas = [120, 95, 140, 80]
2 total = 0
3
4 for venta in ventas:
5     total = total + venta
6
7 print(f"Ventas totales: {total}")
```

Ventas totales: 435

Bucle for con range()

```
1 for numero in range(1, 6):  
2     print(f"Iteración {numero}")
```

Iteración 1
Iteración 2
Iteración 3
Iteración 4
Iteración 5

Bucle `while`

- Un bucle `while` repite instrucciones mientras una condición siga siendo verdadera.
- Es apropiado cuando no sabemos de antemano cuántas iteraciones necesitaremos.

```
1 saldo = 100
2 meses = 0
3
4 while saldo > 0:
5     saldo = saldo - 15
6     meses = meses + 1
7
8 print(f"El saldo se agota después de {meses} meses")
```

El saldo se agota después de 7 meses

break y continue

break

```
1 observaciones = [12, 15, -1, 18, 21]
2
3 for valor in observaciones:
4     if valor < 0:
5         print("Dato inválido, se detiene la revisión.")
6         break
7     print(valor)
```

12

15

Dato inválido, se detiene la revisión.

continue

```
1 ventas = [120, 0, 95, 0, 140]
2
3 for venta in ventas:
4     if venta == 0:
5         continue
6     print(venta)
```

120

95

140

¿Qué veremos hoy?

- Control de flujo y condicionales
- Bucles en Python
- Funciones en Python

¿Por qué usamos funciones?

- Las funciones permiten **reutilizar código**.
- También ayudan a dividir un problema grande en tareas más pequeñas y comprensibles.
- Cuando un bloque de código se repite, suele ser una señal de que conviene encapsularlo en una función.

Crear una función

```
1 def margen_unitario(precio, costo):  
2     return precio - costo  
3  
4  
5 margen_unitario(1500, 920)
```

580

Parámetros y valores de retorno

- Los **parámetros** son los datos que una función necesita para operar.
- La instrucción **return** permite devolver un resultado para usarlo más adelante.
- Una buena función realiza una tarea clara y devuelve una salida fácil de interpretar.

```
1 def calcular_promedio(valores):  
2     return sum(valores) / len(valores)  
3  
4  
5 calcular_promedio([4.2, 5.1, 6.0, 3.8])
```

4.775

Parámetros por defecto

```
1 def precio_final(precio, descuento=0.0):  
2     return precio * (1 - descuento)  
3  
4  
5 print(precio_final(10000))  
6 print(precio_final(10000, descuento=0.15))
```

```
10000.0  
8500.0
```

Ejemplo integrador

```
1 def clasificar_venta(monto):
2     if monto >= 150:
3         return "alta"
4     if monto >= 100:
5         return "media"
6     return "baja"
7
8
9 ventas = [80, 115, 160, 95, 180]
10 resumen = {}
11
12 for venta in ventas:
13     categoria = clasificar_venta(venta)
14     resumen[categoria] = resumen.get(categoria, 0) + 1
15
16 resumen
```

```
{'baja': 2, 'media': 1, 'alta': 2}
```

Ideas clave de la sesión

- Los condicionales permiten tomar decisiones en función de reglas.
- Los bucles permiten automatizar tareas repetitivas.
- Las funciones ayudan a organizar el código y reutilizar procedimientos.
- Estas herramientas son la base para construir análisis de datos más complejos en las próximas semanas.